BUSINESS BRIEF | CHALLENGE 01

# E-commerce Sales & Customer Analytics

A business challenge for turning transactional data into trusted, decision-ready insights with PostgreSQL, SQL and Apache Superset.

| | |
|---|---|
| **Project** | E-commerce Sales & Customer Analytics |
| **Database** | PostgreSQL 16 |
| **Analytics** | SQL and Apache Superset |
| **Dataset** | Reproducible synthetic transaction data |

Prepared as a portfolio project brief. The dataset is synthetic, so findings must be described as illustrative rather than representative of a real company.

# 1. Business context

An e-commerce company records its activity across customers, orders, products, categories and order items. These tables capture individual transactions, but business stakeholders do not yet have a consistent way to answer recurring questions about sales, customer purchasing behavior, product performance and changes over time.

You have been asked to build a reliable SQL analytics layer and a clear dashboard that helps managers understand what is happening in the business, where performance is concentrated, and which questions need further investigation.

# 2. The business challenge

**How can the company turn its raw transactional data into trustworthy, reusable analysis that supports sales and customer decisions?**

The solution should not be a collection of unrelated queries. It should define business metrics consistently, validate the underlying data, produce reusable analytical datasets, and present the results in a dashboard that a non-technical stakeholder can understand.

# 3. Questions the analysis must answer

| Business area | Required questions | Expected evidence |
|---|---|---|
| Sales performance | How much revenue is associated with completed orders? How many orders are completed? What is the average order value? | Documented metric definitions and SQL results. |
| Revenue trends | How do revenue, completed orders and units sold change by month? Are there notable increases or decreases? | Monthly summary query and time-series chart. |
| Product performance | Which products contribute the most revenue and units sold? Which products have never been ordered? | Ranked product table and bar chart. |
| Category performance | Which product categories contribute most to completed-order revenue? | Revenue by category with clear sorting. |
| Customer behavior | Which customers generate the most completed-order revenue? How many customers make repeat purchases? | Customer-level metrics and repeat-purchase definition. |
| Order outcomes | What share of orders are completed, cancelled or returned? | Status counts and rates, with denominator stated. |
| Data reliability | Are there duplicate keys, missing values, invalid relationships or unexpected statuses? | Repeatable quality-check queries and recorded results. |

# 4. Dataset and known constraints

Use the reproducible synthetic dataset generated by `generate_dataset.py`. The expected row counts supplied for the current version are shown below. Verify them against the actual database before analysis; do not treat the expected counts as proof that loading succeeded.

| Table | Purpose | Expected rows |
|---|---|---|
| categories | Product categories | 10 |
| products | Products and category assignment | 100 |
| customers | Customer attributes and signup dates | 300 |
| orders | Customer orders, dates and statuses | 2,000 |

| Table | Purpose | Expected rows |
|---|---|---|
| order_items | Products, quantities and prices per order | 7,027 |

The tables are related through primary and foreign keys: products reference categories; orders reference customers; and order_items reference both orders and products. The data is synthetic and small, so PostgreSQL and SQL are sufficient for this phase. Distributed processing with Spark is not required to solve the current challenge.

# 5. Required business rules

- **Primary sales view:** use orders with status = 'completed' for the main completed-order sales metric.
- **Revenue formula:** sum quantity × order_items.unit_price for eligible order items. Use the transaction price, not the product's current_price, to represent the price recorded at purchase.
- **Average order value:** completed-order revenue divided by the number of completed orders. Use NULLIF or equivalent protection against division by zero.
- **Order outcomes:** show completed, cancelled and returned orders separately. State the denominator for each rate.
- **Returns limitation:** the schema stores a returned status at order level but has no refund amount or partial-return details. Do not claim to calculate actual net revenue or refund value.
- **Customer measures:** define repeat customers consistently, for example customers with at least two completed orders.
- **Interpretation:** distinguish measured results from hypotheses. Synthetic data cannot establish real market behavior.

# 6. Deliverables and acceptance criteria

| Deliverable | Acceptance criteria |
|---|---|
| Verified source database | All five tables exist; actual row counts are recorded; table relationships and sample rows are checked. |
| Data quality checks | SQL checks cover duplicate identifiers, nulls where relevant, valid statuses, orphan records and basic numeric/date validity. Findings are documented. |
| SQL analysis | Separate, readable SQL files answer the sales, trends, product, category and customer questions. JOINs and aggregations are used correctly. |
| Reusable analytics layer | Views or clearly reusable queries expose stable metrics for the dashboard. Metric definitions are documented. |
| Superset dashboard | At least one dashboard includes KPI cards, monthly revenue, category revenue, top products and order status distribution. Charts use actual database results. |
| Reproducibility | README explains setup, database connection, how to run SQL, how to access the dashboard and how to reproduce the analysis. |
| Security and presentation | Secrets are excluded from Git; screenshots are readable; project limitations and synthetic-data status are disclosed. |

# 7. Recommended evaluation rubric

| Area | Weight | What good work looks like |
|---|---|---|
| Correctness of metrics and SQL | 30% | Calculations match documented rules; joins do not inflate totals; edge cases are considered. |
| Data quality and validation | 15% | Checks are repeatable and results are explained. |
| Business insight and interpretation | 20% | Findings answer the stated questions and avoid unsupported claims. |
| Dashboard usability | 20% | Charts are legible, logically ordered, consistent and useful to a business audience. |
| Reproducibility and documentation | 15% | A reviewer can understand and rerun the project without exposing credentials. |

# 8. Out of scope for this phase

- Machine learning or demand forecasting.
- A production cloud deployment or real customer data.
- A distributed Spark pipeline for this small dataset.
- Accounting-grade net revenue, refund and margin calculations without the required source fields.

**Definition of success:** A reviewer can follow the data from the five source tables to documented SQL metrics and then to a Superset dashboard, and can understand both what the results say and what the dataset cannot tell them.